

University
Politehnica
of Bucharest





Computer
Science and
Engineering
Department

Column-Oriented Databases
Cassandra (I)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- Column-Oriented Databases
- Cassandra



Overview

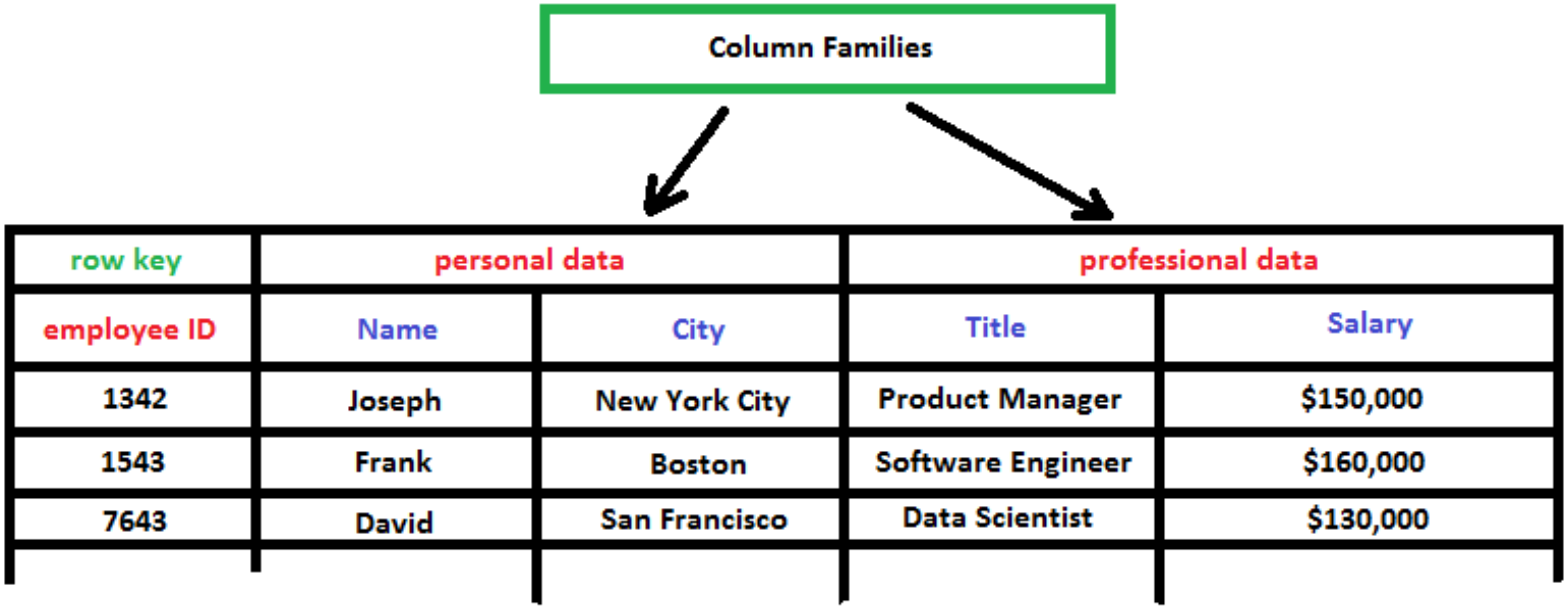
- Column-Oriented Databases
- Cassandra



Column Stores

- Column-Oriented Databases Management Systems (CODBMS) stores data using tables, rows, and columns
- Unlike a relational database, the names and format of the columns can vary from row to row in the same table
- Columns are grouped in Column Families
- CODBMSs are also called Wide Column stores or Columnar Database

Column Stores



row key	personal data		professional data	
employee ID	Name	City	Title	Salary
1342	Joseph	New York City	Product Manager	$150,000
1543	Frank	Boston	Software Engineer	$160,000
7643	David	San Francisco	Data Scientist	$130,000

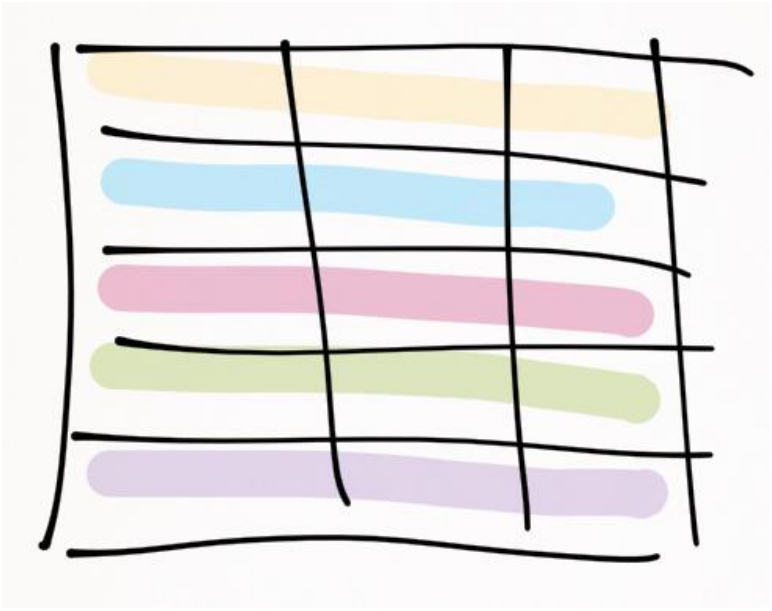


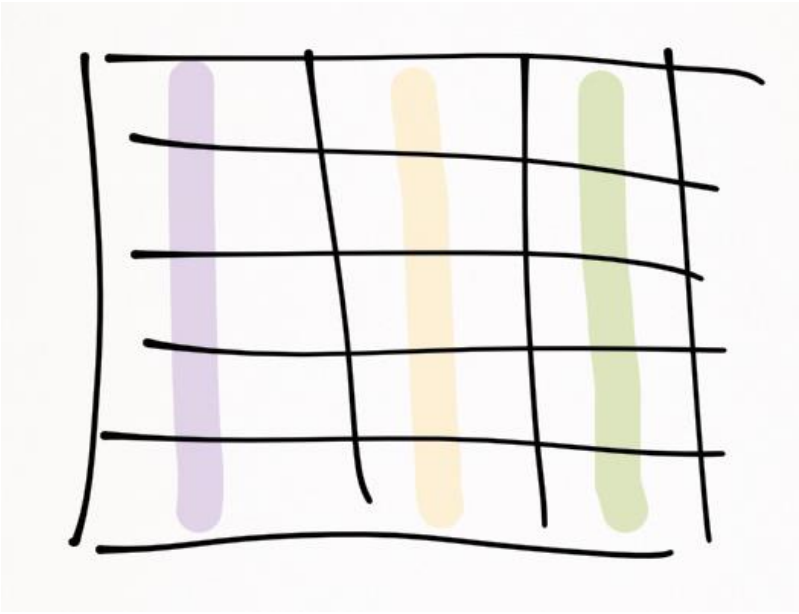
Column Stores

- CODBMS Databases uses one of two approaches to partitioning data on disk:
  - Row-Oriented: partition tables by rows
  - Column-Oriented: partition tables by columns/column families

Column-Oriented Databases

- Partitioning: Row-Oriented vs. Column-Oriented







Row-Oriented partitioning

- Predictability:
  - Tuples address can be predicted without looking at the data
- Locality:
  - Reduce the probability to read the data from
    - Memory cache
    - The same block on disk
- Read operations:
  - To read one attribute the entire tuple must be read

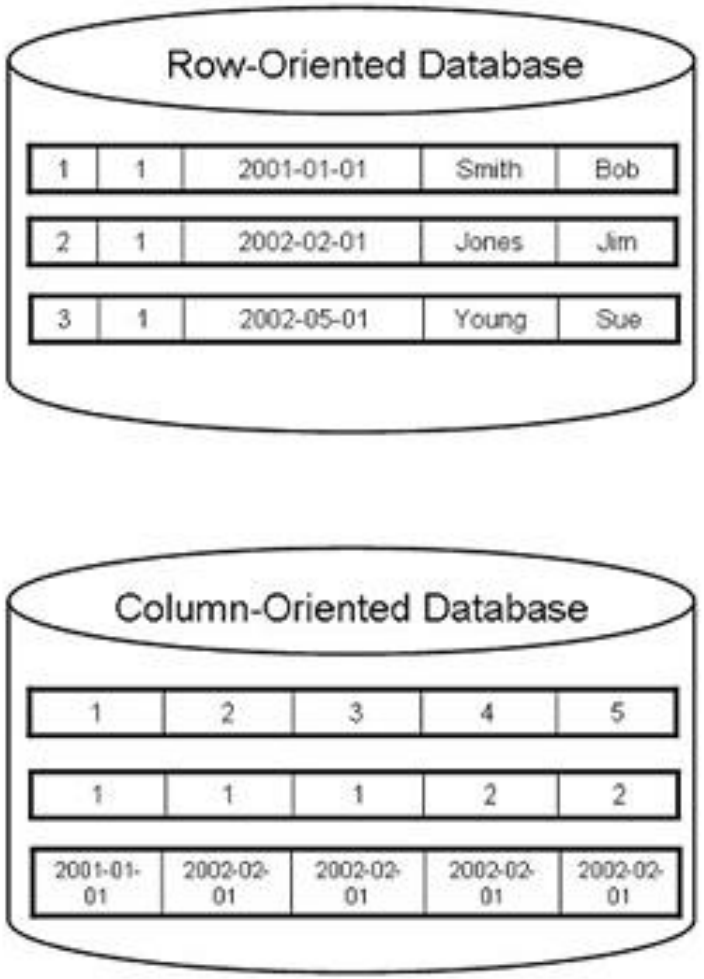


Column-Oriented partitioning

- Predictability:
  - We can predict exactly where we find the data if the data uses fixed length types
  - We cannot predict exactly where we find the data in the case of tuples with variable length
- Locality:
  - The probability to hit the cache (or block) is very high
- Read operations:
  - Read only the needed data

Row-Oriented vs. Column-Oriented

Emp_no	Dept_id	Hire_date	Emp_ln	Emp_fn
1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue
4	2	2003-02-01	Stemle	Bill
5	2	1999-06-15	Aurora	Jack
6	3	2000-08-15	Jung	Laura





Overview

- Column-Oriented Databases
- Cassandra
  - Overview
  - Data Model



Cassandra

- First developed by Facebook to solve the inbox search problem
- The team wanted to address the following problems:
  - Handle huge volumes of data in the form of message copies
  - Reverse indices of messages
  - Many random reads
  - Many simultaneous random writes



Cassandra Features

- Open source
- Distributed
- Decentralized
- Elastically Scalable
- Highly Available and Fault-Tolerant
- Uses Tunable Consistency
- High Performance
- Row-Oriented Partitioning



Cassandra Distributed

- Cassandra can run on multiple machines while appearing to users as a unified whole
- Cassandra design and code base is specifically engineered to:
  - Work across many different machines
  - Optimize performance across multiple data center racks
  - Run a single Cassandra clusters across geographically dispersed data centers



Cassandra Decentralized

- There is no single point of failure
- All the nodes in a Cassandra cluster function exactly the same
  - This is sometimes referred to as “server symmetry”
- Because they are all doing the same thing there can’t be a special host that is coordinating activities, as with the master/slave setup



Cassandra
Elastic Scalable

- Elastic scalability refers to a special property of horizontal scalability
- It means that your cluster can seamlessly
  - Scale up
  - Scale (back) down
- The cluster must be able:
  - To seamlessly accept new nodes
  - New nodes can begin participating by getting a copy of some or all the data
  - New nodes start serving new user requests
    - Without major disruption
    - Reconfiguration of the entire cluster



Cassandra
Elastic Scalable

- There is no need to:
  - Restart any process
  - Change any application queries
  - Manually rebalance the data

- With Elastic Scalability
  - Cassandra will automatically find, start and send work to any new machine added to the cluster



Cassandra
Highly Available and Fault Tolerance

- Failed nodes can be replaced in the cluster with no downtime
- Data can be replicated to multiple data centers to offer improved local performance
- Replication helps to prevent downtime if one data center experiences a catastrophe



Cassandra
Tunable Consistency

- Cassandra allows the administrator to choose the level of consistency
- Cassandra's consistency also uses the replication factor
- The replication factor is a trade-off between how much you want to pay in performance to gain more consistency
- The replication factor value is equal to the number of nodes in the cluster where a modification (insert, update, delete) will propagate to



Cassandra Consistency Level

- The consistency level is a setting that clients must specify on every operation
- The consistency level allows clients to decide how many replicas in the cluster
  - Must acknowledge a write operation in order to be considered
    - Consistent
    - Durable
  - Respond to a read operation in order to be considered successful
- Can be equal to the replication factor:
  - The gain is stronger consistency
  - The cost is synchronous blocking operations that wait for all nodes to be updated and declare success before returning



Cassandra
High Performance

- Cassandra was designed specifically from the ground up to take full advantage of multiprocessor/multi-core machines
- Cassandra can run across many dozens of these machines housed in multiple data centers
- Cassandra scales consistently and seamlessly to hundreds of terabytes
- Cassandra has been shown to perform exceptionally well under heavy load



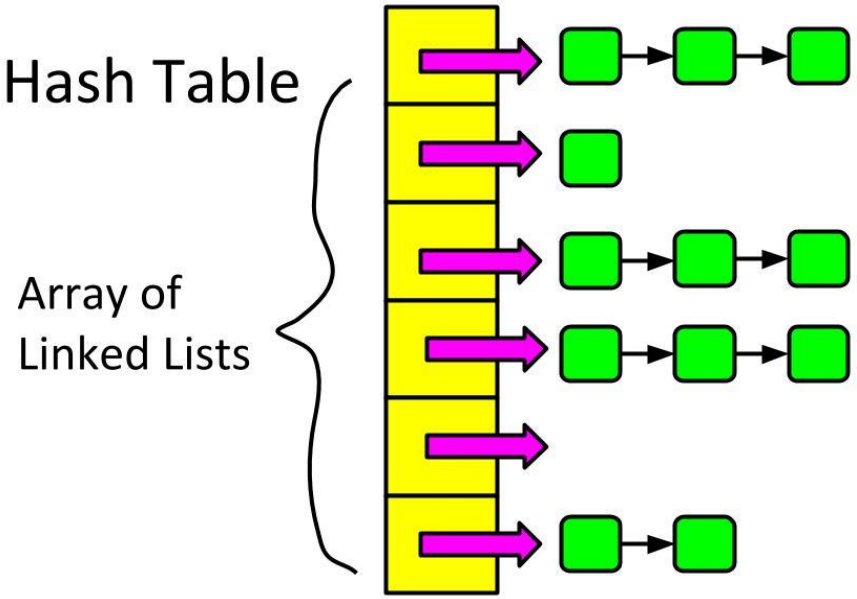
Cassandra
High Performance

- Cassandra consistently can show very fast throughput for writes per second on basic commodity computers
- The commodity computers can run on
  - Physical hardware
  - Virtual machines
- All of Cassandra's desirable properties can be maintained without sacrificing performance as more servers are added



Cassandra
Row-Oriented partitioning

- Cassandra's data model can be described as a partitioned row store
- Data is stored in sparse multidimensional hashtables



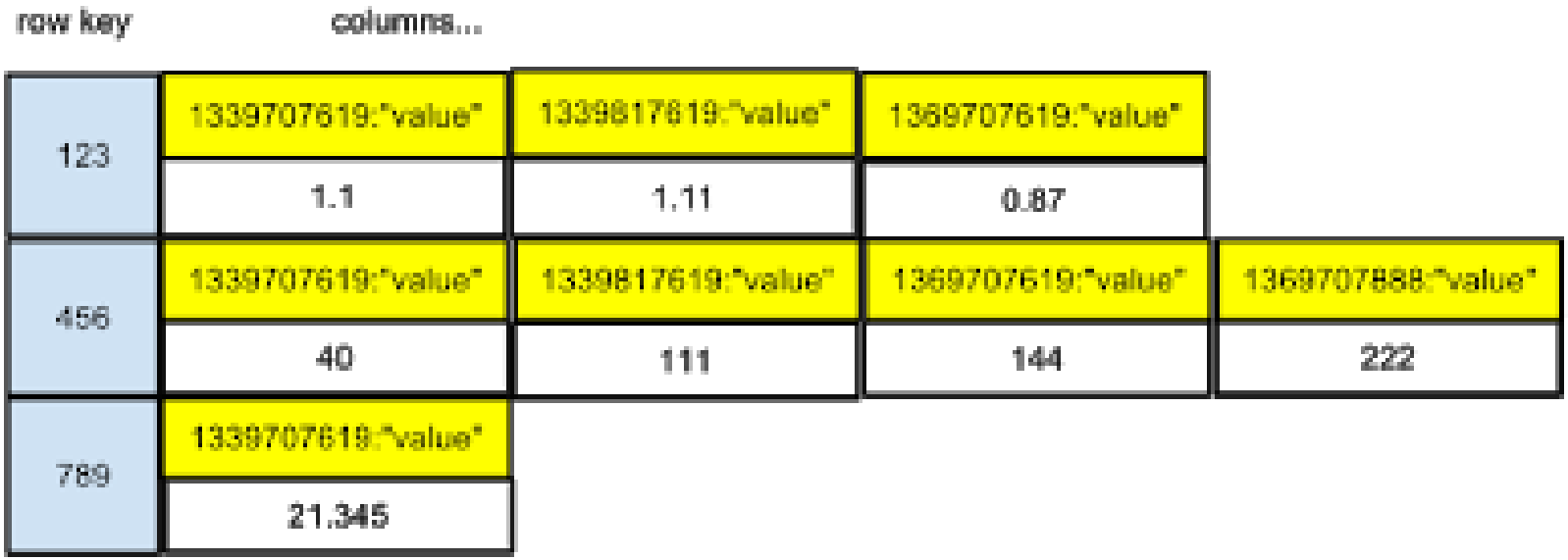


Cassandra
Row-Oriented partitioning

- Sparse means that
  - For any given row you can have one or more columns
  - Each row does not need to have all the same columns as other rows like it (as in a relational model)

- Partitioned means that
  - Each row has a unique key which makes its data accessible
  - The keys are used to distribute the rows across multiple data stores

Cassandra
Row-Oriented partitioning





Cassandra for Applications

- Cassandra is a good fit for
  - Large Deployments
  - Lots of Writes, Statistics, and Analytics
  - Geographical Distribution
  - Evolving Applications



Cassandra for Applications
Large Deployments

- Cassandra is a good fit
  - If the application is expected to require dozens of nodes
  - For applications that needs high availability
  - For applications that are geographical distributed



Cassandra for Applications
Lots of Writes

- Before choosing a database, consider your application from the perspective of the ratio of reads to writes
- Cassandra is optimized for excellent throughput on writes
- One of Cassandra's primary feature is the ability to handle application workloads that require high performance at significant write volumes with many concurrent client threads



Cassandra for Applications Statistics

- Cassandra is a good choice for storing
  - User activity updates
  - Social network usage
  - Recommendations
  - Reviews
  - Application statistics

- Cassandra is also a good choice when the application involve lots of writing with less predictable read operations

- When updates can occur unevenly with sudden spikes



Cassandra Analytics

- Cassandra has been used to create a variety of analysis applications, e.g.
  - Windowed time-series store,
  - An inverted index for document searching
  - A distributed job priority queue, etc



Cassandra for Applications Geographical Distribution

- Cassandra has out-of-the-box support for geographical distribution of data
- Cassandra can be easily configured to replicate data across multiple data centers
- Cassandra could be a great fit if you have a globally deployed application that could see a performance benefit from putting the data near the user



Cassandrafor for Applications Evolving Applicaitons

- Cassandra might be a good fit if the application is evolving rapidly given its support for flexible schemas
- Cassandra makes it easy to keep your database in step with application changes as you rapidly deploy



Overview

- Column-Oriented Databases
- Cassandra
  - Overview
  - Data Model



Cassandra – Data Model

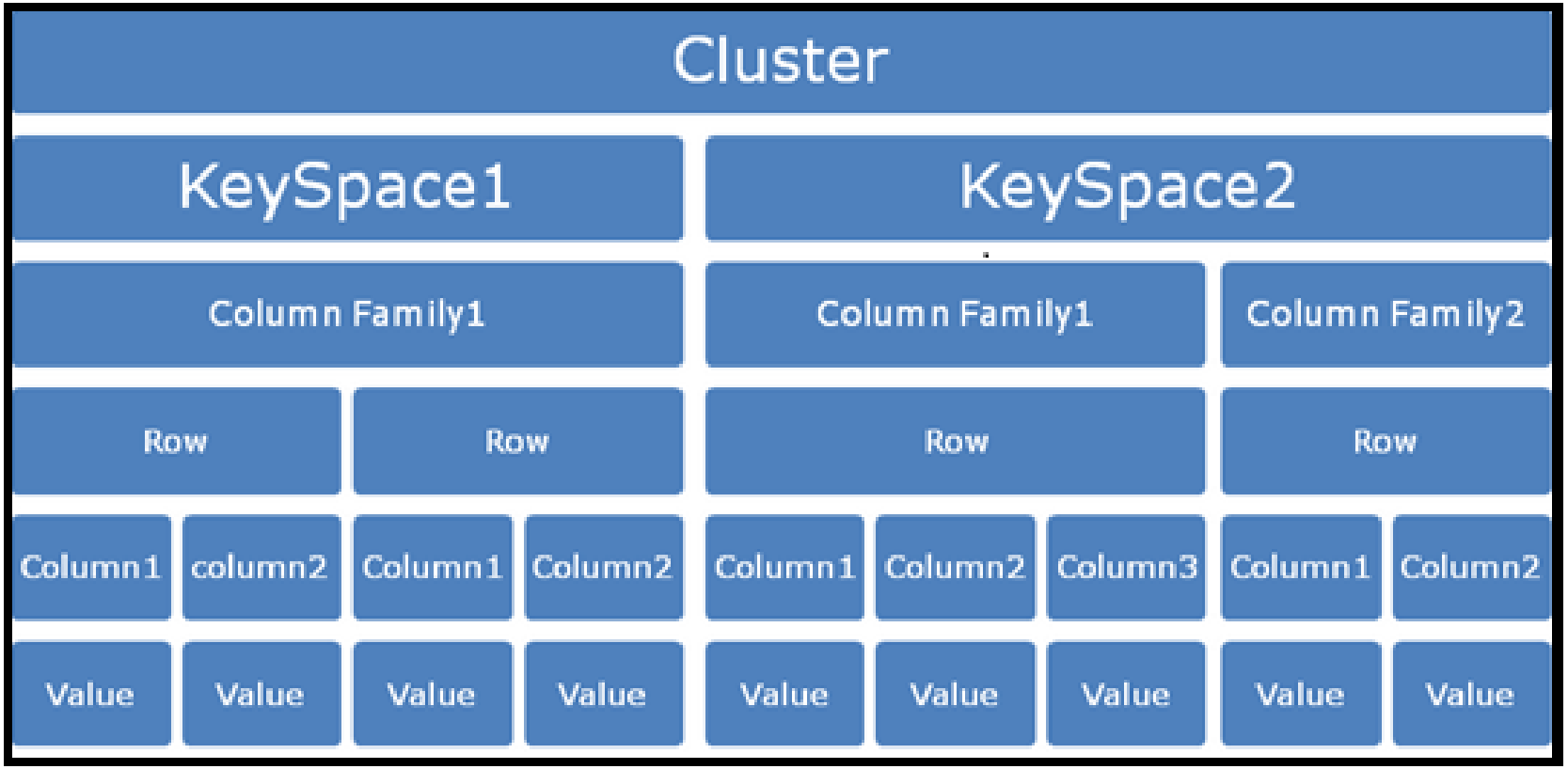
- Cassandra data model contains the following structures:
  - Cluster
  - Keyspace
  - Table



Cassandra – Data Model

- An instance of Cassandra typically consists of only one table
- A table represents a distributed multidimensional map indexed by a key
- A table is structured by the following dimensions (bottom-up view)
  - Rows
  - Row Key (Partition Key)
  - Column-families
  - Columns
  - Super-columns

Cassandra – Data Model





Cassandra – Data Model
Rows

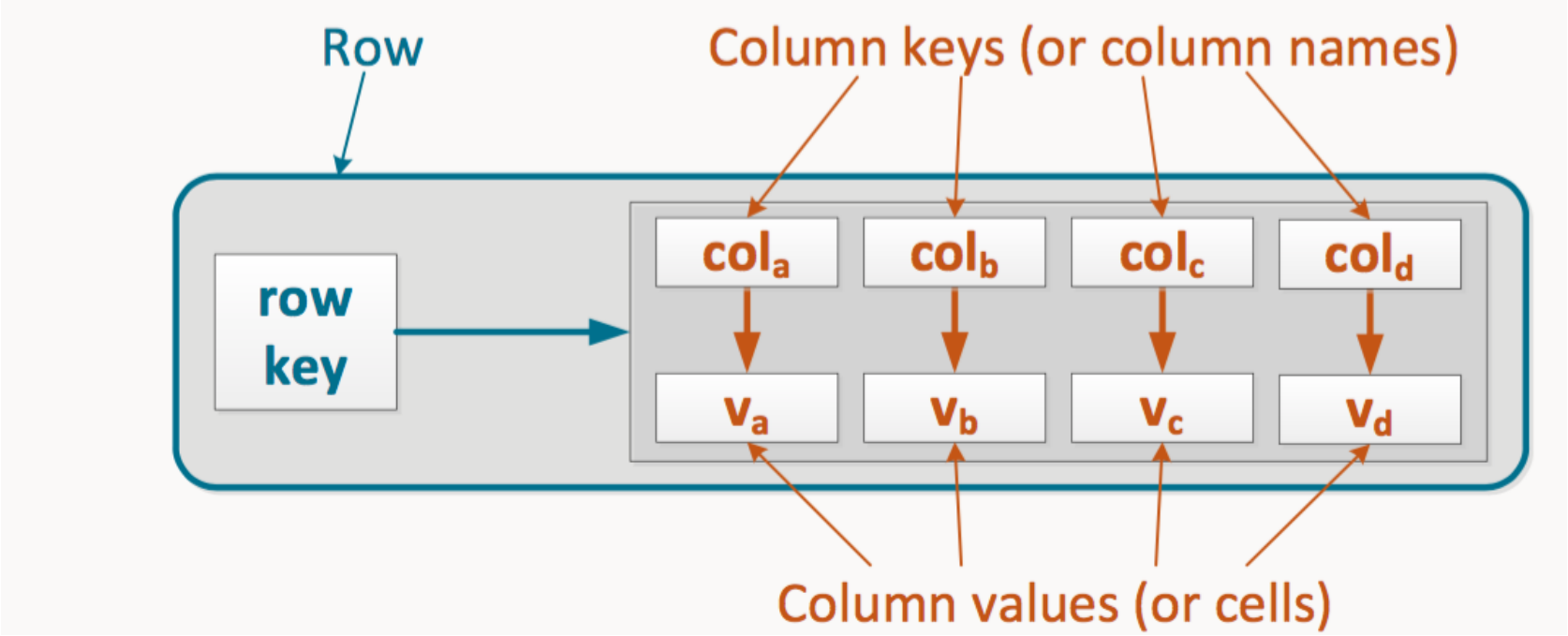
- A row is the smallest unit that stores related data in Cassandra
- Rows are identified by a string-key of arbitrary length
- Operations on rows are atomic per replica no matter how many columns are being read or written
- A row key uniquely identifies a row in a column family
- A row stores pairs of column keys and column values
- A column key uniquely identifies a column value in a row
- A column value stores one value or a collection of values



Cassandra – Data Model
Rows

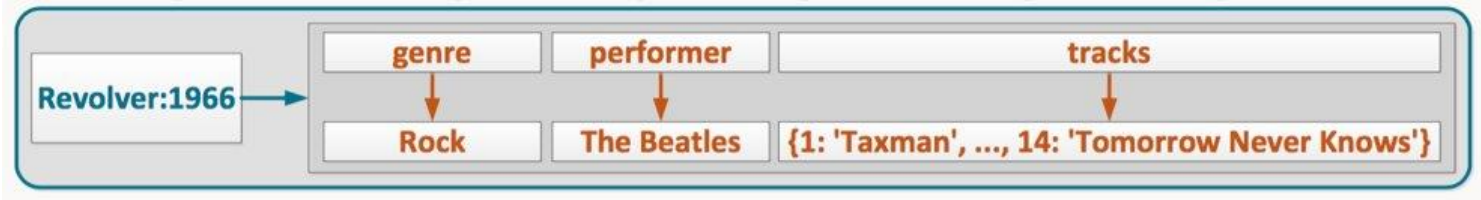
- Rows may be described as skinny or wide
  - Skinny row:
    - Has a fixed number of column keys
    - Relatively small number of column keys
  - Wide row:
    - Has a relatively large number of column keys (hundreds or thousands)
    - The number of column keys may increase as new data values are inserted

Cassandra – Data Model
Rows

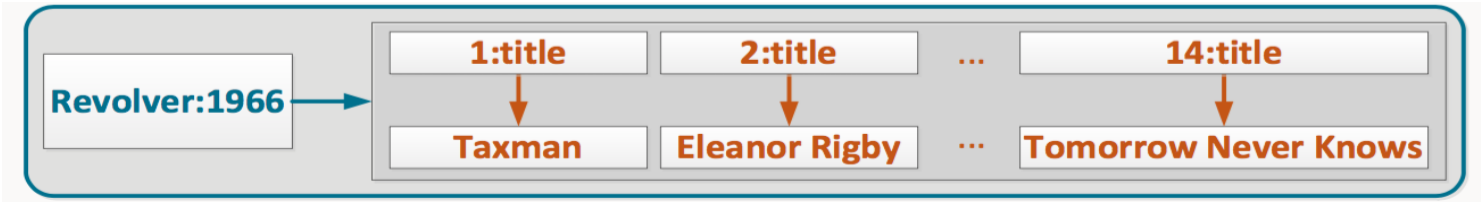


Cassandra – Data Model
Keys

- Partition Key
  - Simple row key
  - Composite row key
    - Multiple components can form a row key
    - The components are separated by colon



- Column Keys
  - Simple column key
  - Composite column key:
    - Multiple components can form a column key
    - The components are separated by colon

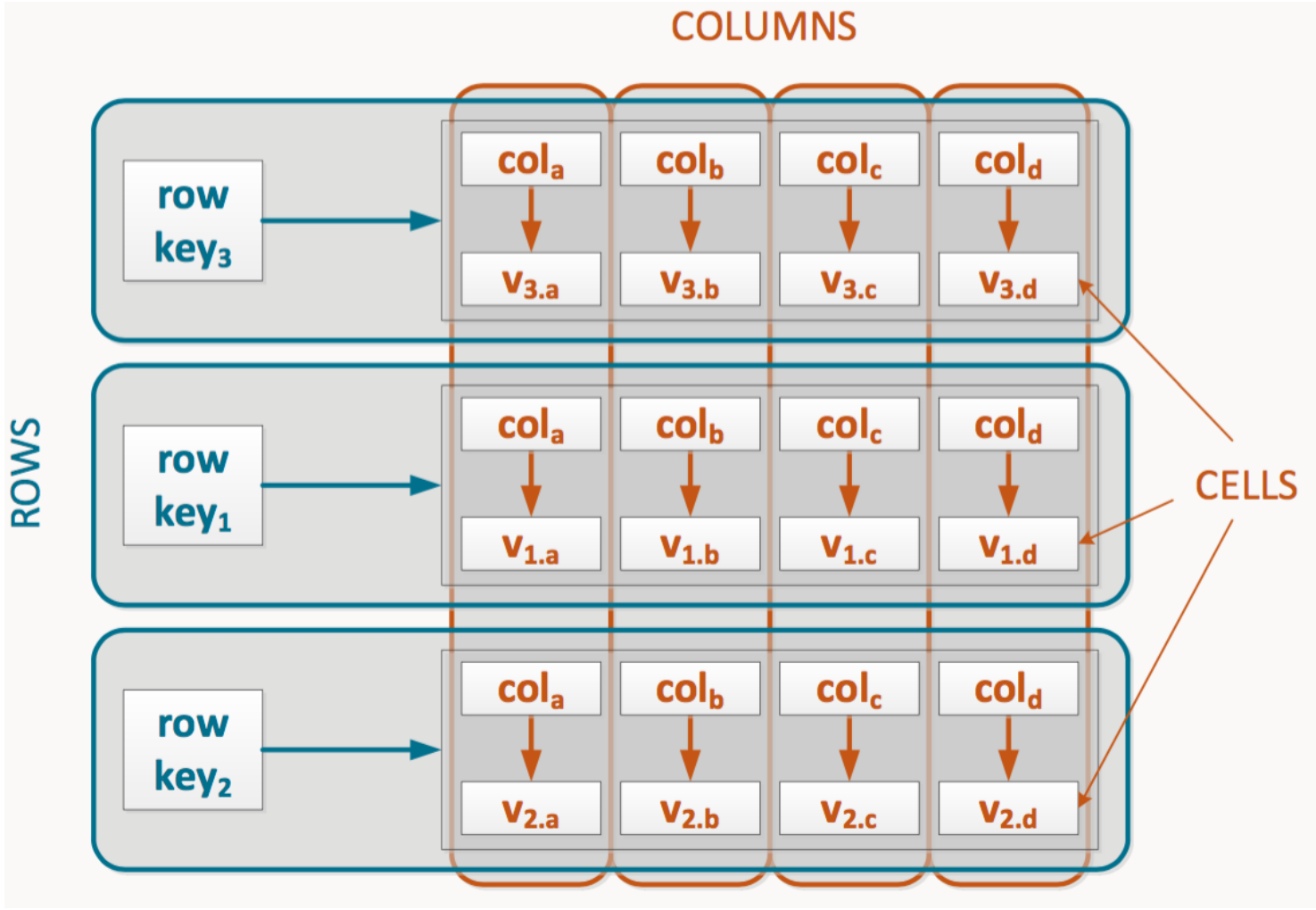




Cassandra – Data Model Column Families

- Are a set of rows with a similar structure
- Can occur in an arbitrary number per row
- The number of column families per table is not limited
- It is expected that only a few of them are specified at the table creation time:
  - These must be defined in advance before a cluster of servers comprising a Cassandra instance is launched
- Consists of columns and super columns
  - Can be added dynamically (i.e. at runtime) to column families
  - Are not restricted in number
- In analogy with relational databases
  - A standard column family is as a table
  - Each key-value pair being a row

Cassandra – Data Model
Column Families





Cassandra – Data Model Columns

- Columns store several values per row
  - The values are identified by a timestamp
- Columns must have a name
  - The name acts as the column key
- Each row in a table can have a different number of columns
  - A table can be thought of as a rectangle
- Client applications may specify the ordering of columns within
  - A column family
  - Super column
- The ordering can either be by name or by timestamp



Cassandra – Data Model
Super columns

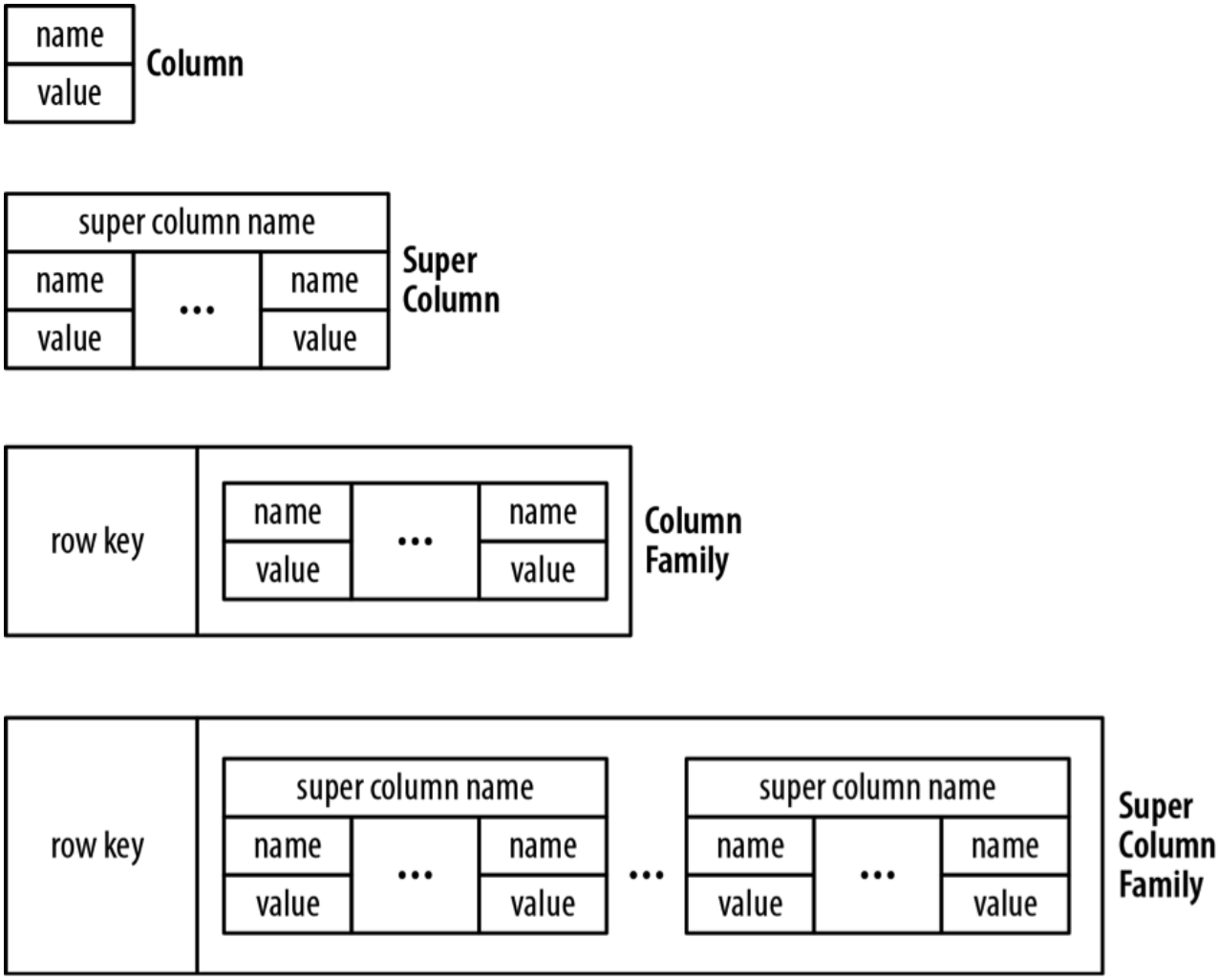
- Super columns
  - Are column containers sorted by their names
  - Are tuples (pairs) that consist of a key-value pair
    - The key is mapped to a value that is a set of columns
    - Can have an arbitrary number of columns associated with them
- The number of columns per super column may differ per row
- The columns of a super column
  - Must have a name
  - The name acts as a unique identifier
  - Can be referenced and sorted by their row key



Cassandra – Data Model
Super column families

- When rows contain column families, then the collection is called a super column family
- A super column family is a tuple (pair) that consists of a key-value pair
- The key is mapped to a value that are column families
- In analogy with relational databases, a super column family is something like a view on several tables (JOINS)
- It can also be a map of tables

Cassandra – Data Model





Cassandra – Data Model Clusters

- The outermost structure in Cassandra is the cluster

- Cassandra database is specifically designed to be distributed over several machines operating together
  - These machines appear as a single instance to the end user
  - The cluster is built on these machines

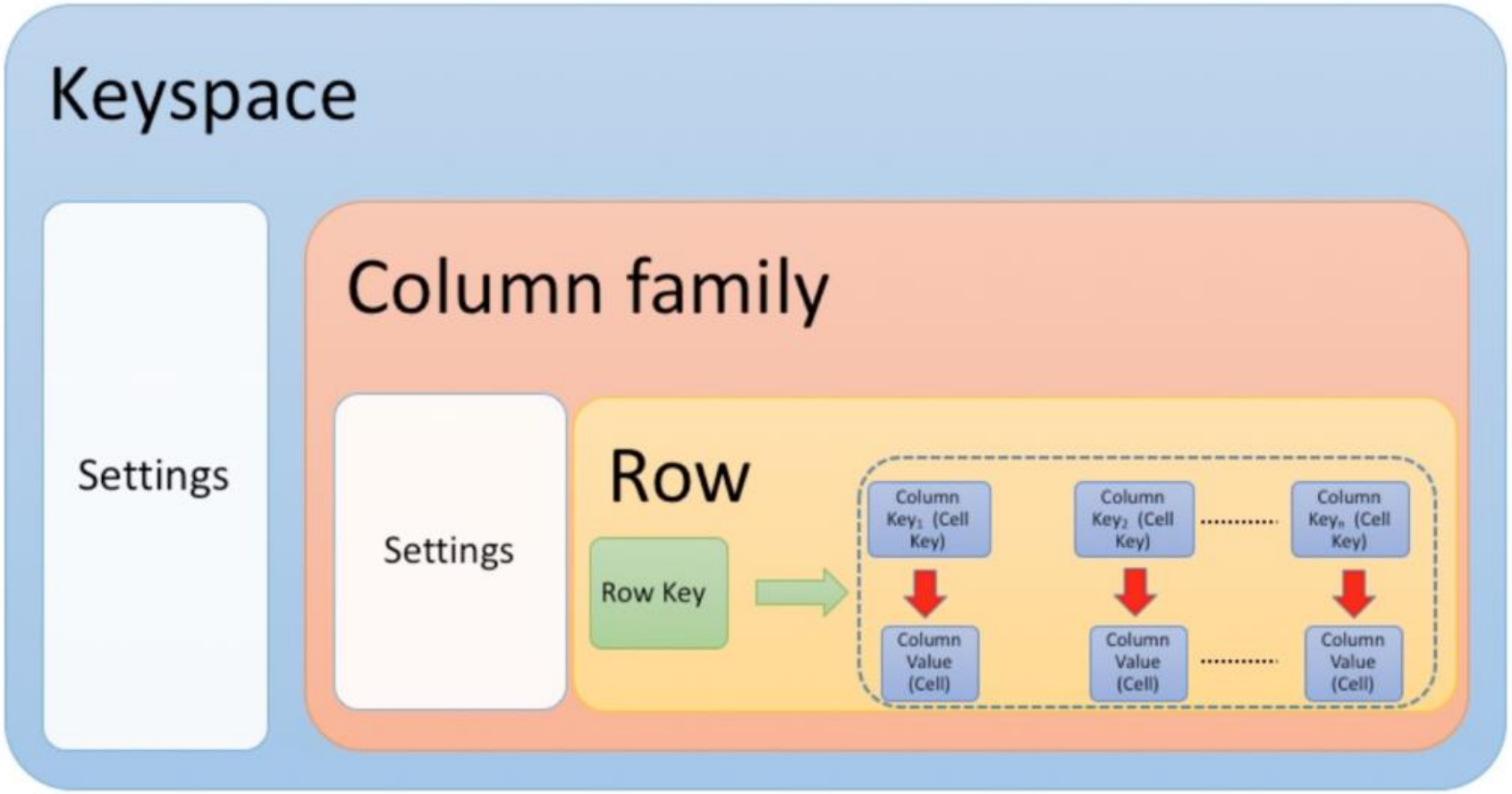
- The cluster is sometimes called the ring
  - Cassandra assigns data to nodes in the cluster by arranging them in a ring



Cassandra – Data Model Keyspaces

- A cluster is a container for keyspaces.
- A keyspace is the outermost container for data in Cassandra
- In analogy with relational databases:
  - The keyspace corresponds closely to a relational database (schema)
  - In the same way that a database (schema) is a container for tables in the relational model, a keyspace is a container for tables in the Cassandra data model.
- A keyspace has a name and a set of attributes that define keyspace-wide behavior.

Cassandra – Data Model
Keyspaces



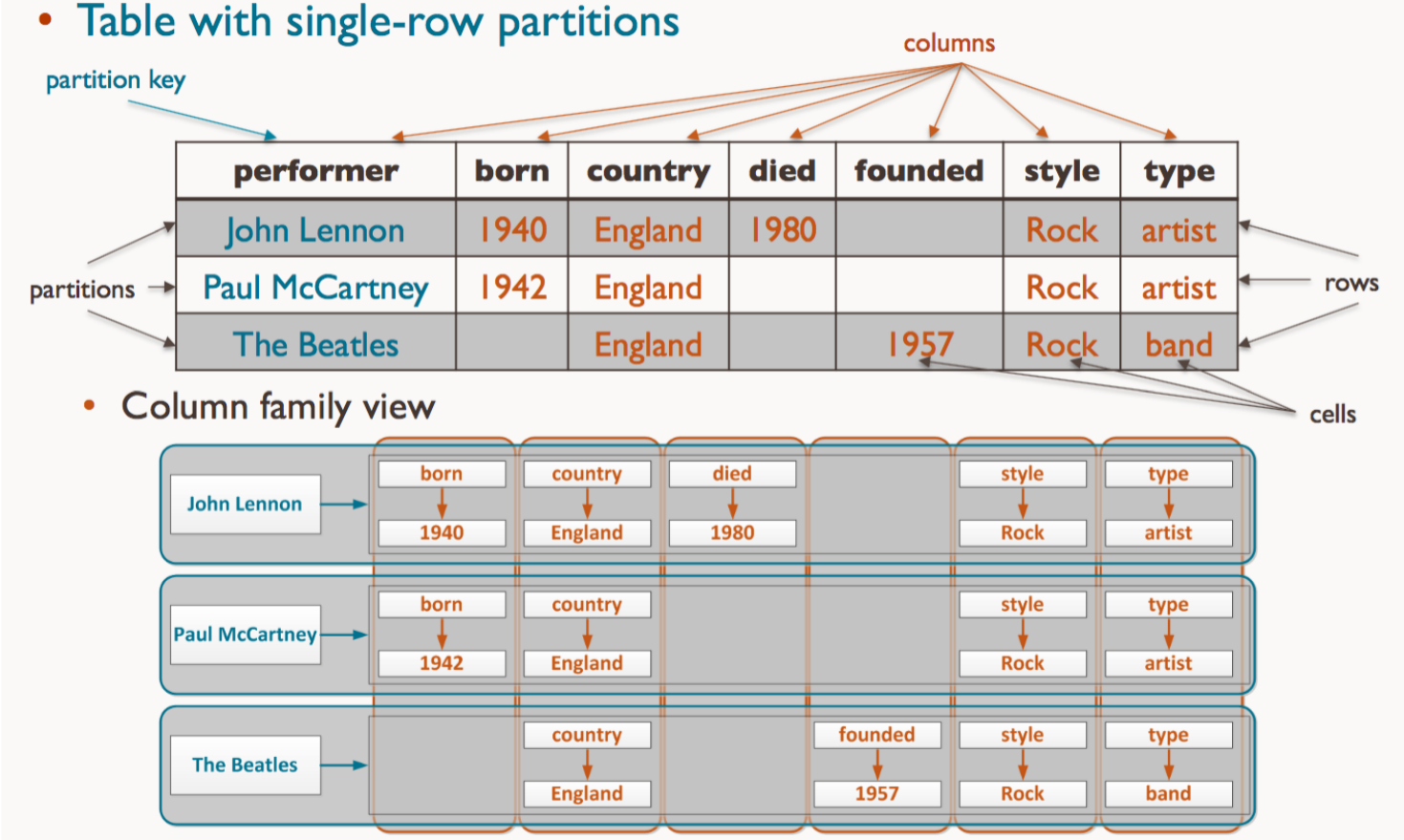


Cassandra – Data Model Tables

- Model vs. Implementation
- A column family is created using the **CQL CREATE TABLE** command
- A table is a container for an ordered collection of rows (records) as defined in a Cassandra Database
- Each row (record) is itself an ordered collection of columns values
- The ordering is determined by the columns
  - When you write data to a table in Cassandra, you specify values for one or more columns
  - The column names are identified as (column) keys
  - At least one of the values you specify must be a row key that serves as the unique identifier for that record

Cassandra – Data Model Tables

- Table with single-row partitions



Cassandra – Data Model Tables

- Table with multi-row partitions

